

Ticket to ride? Data study on the road.



Dave Cuthbert

Jan 19 · 10 min read



It's nearly the end of the month as I write this. A long drive awaits. Will the police be on the road, out in force to meet monthly ticket quotas? Do police departments really have ticket quotas leading to increased traffic stops for unhappy motorists? It's an age old question. Google quickly provides strong statements from believers and skeptics on both sides of the debate.

There's opinion, and there's also some data too, and that's the real subject of this post. The state Maryland has contributed a collection of

traffic violation data to data.gov. It's just one data set among more than 160,000 others on that site that are freely available for the curious. The Maryland Traffic Data does shed light on the opening question. More importantly, working with this data set also provided useful lessons and reminders for researchers working with any data set.

Getting Started

Any good study begins with a question. In this case let's start with the obvious one, 'are there more tickets issued at the end of the month'? If so that suggests police officers could be playing catch up for quotas. It would also confirm anecdotal stories about when tickets are issued.

OK, we have a good question. Now it's time to hunt for a good data set to shed light on it. Data.gov, mentioned above, has a massive collection of data sets on many different topics. Fortunately along with the data itself, data.gov provides a little meta-data for each dataset. The metadata varies, but generally it provides some useful information about the contents of each collection of data. In this case the metadata tells us that there is a csv file from Montgomery County in the state of Maryland that reportedly "contains traffic violation information from all electronic traffic violations issued in the County".

The screenshot shows the Data Catalog interface for the 'Comma Separated Values File' dataset. The page includes a navigation bar with 'DATA CATALOG', 'Datasets', and 'Organizations'. The breadcrumb trail indicates the location: 'Montgomery County of Maryland / Traffic Violations / data.montgomerycountymd.gov'. The dataset title is 'Comma Separated Values File', with a 'Download' button. The URL is 'https://data.montgomerycountymd.gov/api/views/4mse-ku6q/rows.csv?accessType=DOWNLOAD'. The 'Resource Quality' section shows a rating of 5 stars. The 'About this Resource' section provides details: 'Last updated' and 'Created' are both February 4, 2015; 'Format' is 'Comma Separated Values File'; and 'License' is 'License not specified'. A 'Show more' link is available. On the left, there are links to download the file in RDF, JSON, or XML format, and social media sharing options for Google+, Twitter, and Facebook.

DATA CATALOG [/ Datasets](#) [Organizations](#) [?](#)

[/](#) [Montgomery County of Maryland](#) / Traffic Violations / data.montgomerycountymd.gov

Comma Separated Values File [Download](#)

URL: <https://data.montgomerycountymd.gov/api/views/4mse-ku6q/rows.csv?accessType=DOWNLOAD>

Resource Quality

Rating: ☆ ☆ ☆ ☆ ☆

About these ratings...

From the dataset abstract

This dataset contains traffic violation information from all electronic traffic violations issued in the County. Any information that can be used to uniquely identify the vehicle, the...

Source: Traffic Violations

False

Resources

Comma Separated Values ...

RDF File

JSON File

XML File

Share on Social Sites

Google+

Twitter

Facebook

About this Resource

Last updated	February 4, 2015
Created	February 4, 2015
Format	Comma Separated Values File
License	License not specified

[Show more](#)

Great! Now it's time to download the file and rename it to something more convenient. Once we have the file, the next step, a key step in every data analysis, is to validate the data. It's a big data set and stored

as a giant csv table so I decided to load it up into an SQLite database to make checking the data and working with it more convenient. In order to do convert the file from cvs to a database, I had to first inspect the file format and prepare some scripts to clean and load the data.

The datafile is about 375MB in size has just under 1.1 million rows, so lots of data to work with! The linux `head` command revealed a header row and some variety in the data encoded in the file.

```
File Edit View Bookmarks Settings Help
traffic: bash - Konsole
[dave@localhost traffic]$ head traffic.csv
Date Of Stop,Time Of Stop,Agency,Subagency,Description,Location,Latitude,Longitude,Accident,Belts,Personal Injury,Property Damage,Fatal,Commercial License,HACMAr,Commercial Vehicle,Alcohol,Work Zone,State,VehicleType,Year,Make,Model,Color,Violation Type,Charge,Article,Contributed To Accident,Race,Gender,Driver City,Driver State,DL State,Arrest Type,Geolocate
09/24/2013,17:11:00,MCP,"3rd district, Silver Spring",DRIVING VEHICLE ON HIGHWAY WITH SUSPENDED REGISTRATION,8804 FLOWER
AVE,,No,No,No,No,No,No,No,No,No,No,MD,02 - Automobile,2008,FORD,4S,BLACK,Citation,13-401(h),Transportation Article,No,
BLACK,M,TAROMA PARK,MD,MD,A - Marked Patrol.
12/20/2012,00:41:00,MCP,"2nd district, Bethesda",DRIVING WHILE IMPAIRED BY ALCOHOL,NORFOLK AVE / ST ELMO AVE,39.9835782
-77.09310515,No,No,No,No,No,No,No,No,No,No,MD,02 - Automobile,2005,AUDI,4S,GRAY,Citation,21-902(b1),Transportation Article,
No,WHITE,M,DERWOOD,MD,MD,A - Marked Patrol,"(39.9835782, -77.09310515)"
07/20/2012,23:12:00,MCP,"5th district, Germantown",FAILURE TO STOP AT STOP SIGN,WISTERIA DR & WARING STATION RD,39.16180
98166667,-77.25358095,No,No,No,No,No,No,No,No,No,No,MD,02 - Automobile,2002,TOYOT,4S,RED,Citation,21-707(a),Transportatio
n Article,No,ASIAN,F,GERMANTOWN,MD,MD,A - Marked Patrol,"(39.1618098166667, -77.25358095)"
03/19/2012,16:10:00,MCP,"2nd district, Bethesda",DRIVER USING HANDS TO USE HANDHELD TELEPHONE WHILE MOTOR VEHICLE IS IN M
OTION,CLARENDON RD & ELM ST. W/38.982730733333,-77.100756166667,No,No,No,No,No,No,No,No,No,No,VA,02 - Automobile,1996
,HONDA,CIVIC,SILVER,Citation,21-1124.2(d2),Transportation Article,No,HISPANIC,M,ARLINGTON,VA,VA,A - Marked Patrol,"(38.9
827307333333,-77.100756166667)"
12/01/2014,12:52:00,MCP,"4th district, Gaithersburg / Montgomery Village",FAILURE STOP AND YIELD AT THRU HWY,CHRISTOPHER
AVE/MONTGOMERY VILLAGE AVE,39.162888333333,-77.229088333333,No,No,No,Yes,No,No,No,No,No,No,MD,02 - Automobile,2001,HO
NDA,ACCORD,SILVER,Citation,21-403(b),Transportation Article,No,BLACK,F,UPPER MARLBORO,MD,MD,A - Marked Patrol,"(39.16288
8333333,-77.229088333333)"
06/09/2012,21:19:00,MCP,"3rd district, Silver Spring",OCCUPANT UNDER 16 NOT RESTRAINED BY SEATBELT,2068 HARLEQUIN TERRAC
E,39.06914295,-76.969678066667,No,No,No,No,No,No,No,No,No,No,MD,02 - Automobile,2004,CHEVROLET,IMPALA,SILVER,Citation,2
2-412.3(b),Transportation Article,No,WHITE,F,SILVER SPRING,MD,MD,A - Marked Patrol,"(39.06914295, -76.969678066667)"
09/11/2012,21:47:00,MCP,"4th district, Wheaton",PERSON DRIVING MOTOR VEHICLE ON HIGHWAY OR PUBLIC USE PROPERTY ON REVOKED
LICENSE AND PRIVILEGES,TWIG RD AT GOOD HOPE RD,39.09619885,-76.98696215,No,Yes,No,No,No,No,No,No,No,No,MD,02 - Automobi
le,2009,DODGE,CHARGER,BLACK,Citation,16-303(d),Transportation Article,No,BLACK,M,SILVER SPRING,MD,MD,A - Marked Patrol,"
(39.09619885, -76.98696215)"
11/04/2012,01:14:00,MCP,"4th district, Wheaton",DRIVING UNDER THE INFLUENCE OF ALCOHOL PER SE,RANDOLPH RD & SPRINGTREE D
R,39.0583676,-77.047982566667,No,No,No,No,No,No,No,No,No,No,MD,02 - Automobile,2003,FORD,TAURUS,WHITE,Citation,21-902(a)
Transportation Article,No,BLACK,M,SILVER SPRING,MD,MD,A - Marked Patrol,"(39.0583676, -77.047982566667)"
08/27/2012,00:04:00,MCP,"5th district, Germantown",PERSON DRIVING MOTOR VEHICLE WHILE LICENSE SUSPENDED UNDER 17-106,2
6-204, 26-206, 27-103*,GERMANTOWN RD&CLOPPER,39.161286166667,-77.28378918333333,No,No,No,No,No,No,No,No,No,No,MD,02 - Au
tomobile,2004,INFINITI,QX4,SILVER,Citation,16-303(h),Transportation Article,No,WHITE,H,GERMANTOWN,MD,MD,A - Marked Patro
l,"(39.161286166667, -77.283789183333)"
[dave@localhost traffic]$
```

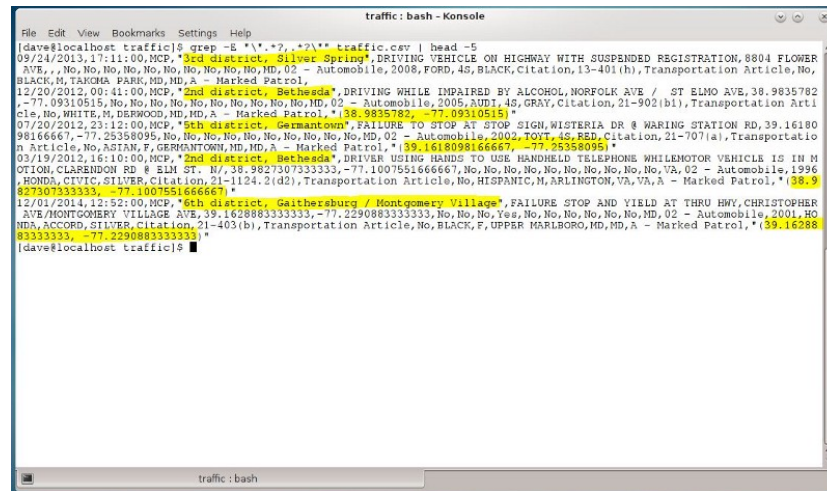
Running a few checks with `grep` highlighted some problems with the data. The file was uploaded in 2015, which made it easy to see that the year field contained thousands of entries with obviously bad dates.

There were missing dates, very early dates and too late dates. Rather than simply drop rows with improbable dates, I added some WHERE clauses to my SQL queries to restrict the results to ranges of dates that were at least plausible.

```
File Edit View Bookmarks Settings Help
tmp: bash - Konsole <2>
[dave@localhost tmp]$ grep '1992,' traffic.csv
06/19/2016,19:39:00,MCP,Headquarters and Special Operations,EXCEEDING THE POSTED SPEED LIMIT OF 55 MPH,370/DNADY GROVE RD,39.128858333333,-77.184001666667,No,No,No,No,No,No,No,No,No,No,MD,02 - Automobile,1992,CADIL,ESCAPE,TAN,Warning,21-801(i),Transportation Article,No,BLACK,H,UPPER MARLBORO,MD,MD,A - Marked Patrol,"(39.128858333333, -77.184001666667)"
[dave@localhost tmp]$ grep ',1898,' traffic.csv
03/19/2011,11:10:00,MCP,"4th district, Wheaton",FAILURE OF INDIVIDUAL DRIVING ON HIGHWAY TO DISPLAY LICENSE TO UNIFORMED POLICE ON BRAND,NEW HANSHIRE AVE / BAY HILL RD,39.128291666667,-76.991071,No,No,No,No,No,No,No,No,No,No,MD,02 - Automobile,1898,FORD,ESCORT,SILVER,Warning,16-112(c),Transportation Article,No,WHITE,F,SILVER SPRING,MD,MD,A - Marked Patrol,"(39.128291666667, -76.991071)"
03/19/2011,11:10:00,MCP,"4th district, Wheaton",FAILURE TO ATTACH VEHICLE REGISTRATION PLATES AT FRONT AND REAR,NEW HANSHIRE AVE / BAY HILL RD,39.128291666667,-76.991071,No,No,No,No,No,No,No,No,No,No,MD,02 - Automobile,1898,FORD,ESCORT,SILVER,Warning,16-112(c),Transportation Article,No,WHITE,F,SILVER SPRING,MD,MD,A - Marked Patrol,"(39.128291666667, -76.991071)"
[dave@localhost tmp]$ grep ',1930,' traffic.csv
07/26/2014,20:01:00,MCP,"4th district, Wheaton",DRIVER FAILURE TO OBEY PROPERLY PLACED TRAFFIC CONTROL DEVICE INSTRUCTIONS,17811 GEORGIA,39.149221666667,-77.066476666667,No,No,No,No,No,No,No,No,No,No,MD,03 - Station Wagon,1930,FORD,TE,BLACK,Warning,21-301(a),Transportation Article,No,ASIAN,H,TEST,MD,MD,A - Marked Patrol,"(39.149221666667, -77.066476666667)"
07/26/2014,20:01:00,MCP,"4th district, Wheaton",DRIVER CHANGING LANE WHEN UNLAWFUL,17811 GEORGIA,39.149221666667,-77.066476666667,No,No,No,No,No,No,No,No,No,No,MD,03 - Station Wagon,1930,FORD,TE,BLACK,Warning,21-301(b),Transportation Article,No,ASIAN,H,TEST,MD,MD,A - Marked Patrol,"(39.149221666667, -77.066476666667)"
07/26/2014,20:01:00,MCP,"4th district, Wheaton",FAILURE TO DRIVE VEHICLE ON RIGHT HALF OF ROADWAY WHEN PROHIBITED,17811 GEORGIA,39.149221666667,-77.066476666667,No,No,No,No,No,No,No,No,No,No,MD,03 - Station Wagon,1930,FORD,TE,BLACK,Warning,21-301(a),Transportation Article,No,ASIAN,H,TEST,MD,MD,A - Marked Patrol,"(39.149221666667, -77.066476666667)"
07/26/2014,20:01:00,MCP,"4th district, Wheaton",DRIVER FAILURE TO STOP AT STOP SIGN LINE,17811 GEORGIA,39.149221666667,-77.066476666667,No,No,No,No,No,No,No,No,No,No,MD,03 - Station Wagon,1930,FORD,TE,BLACK,Warning,21-707(a),Transportation Article,No,ASIAN,H,TEST,MD,MD,A - Marked Patrol,"(39.149221666667, -77.066476666667)"
[dave@localhost tmp]$
```

There were other problems with the raw data. This was a csv file. The extension 'csv' (mostly) means 'character separated values' but often people change the 'c' to mean 'comma'. When that happens, commas are considered reserved characters that are only used to separate the fields in each row. In this file, that didn't happen. Commas were used

both within fields as normal punctuation and as field separators. Even worse, use of commas within fields was inconsistent. That created a real mess for parsing the file. For example, in some rows the sub-agency (usually a geographic designation) had a comma in its name. Other fields occasionally had internal commas too. When they were present in a row, geographic coordinates were recorded as “(latitude, longitude)”



```
traffic: bash - Konsole
File Edit View Bookmarks Settings Help
[dave@localhost traffic]$ grep -E '\',.*? traffic.csv | head -5
09/24/2013,17:11:00,NCP,"2nd district, Silver Spring",DRIVING VEHICLE ON HIGHWAY WITH SUSPENDED REGISTRATION,8804 FLOWER
AVE,,No,No,No,No,No,No,No,No,No,No,No,No,MD,02 - Automobile,2008,FORD,45,BLACK,Citation,13-401(h),Transportation Article,No,
BLACK,M,TANCHA PARK,MD,MD,A - Marked Patrol,
12/20/2012,09:41:00,NCP,"2nd district, Bethesda",DRIVING WHILE IMPAIRED BY ALCOHOL,NORFOLK AVE / ST ELMO AVE,38,9835782
,-77,09310515,No,No,No,No,No,No,No,No,No,No,No,No,MD,02 - Automobile,2005,AUDI,45,GRAY,Citation,21-902(b),Transportation Arti
cle,No,WHITE,M,DERWOOD,MD,MD,A - Marked Patrol,"(38,9835782;-77,09310515)"
07/20/2012,23:12:00,NCP,"8th district, Germantown",FAILURE TO STOP AT STOP SIGN,WISTERIA DR & WARING STATION RD,39,16180
98166667,-77,25358095,No,No,No,No,No,No,No,No,No,No,No,No,MD,02 - Automobile,2002,TOY,45,RED,Citation,21-707(a),Transportatio
n Article,No,ASIAN,F,GERMANTOWN,MD,MD,A - Marked Patrol,"(39,1618098166667;-77,25358095)"
03/19/2012,16:10:00,NCP,"2nd district, Bethesda",DRIVER USING HANDS TO USE HANDHELD TELEPHONE WHILE MOTOR VEHICLE IS IN M
OTION,CLARENDON RD & ELM ST. N/,38,9827307333333,-77,1007551666667,No,No,No,No,No,No,No,No,No,No,No,VA,02 - Automobile,1996
,HONDA,CIVIC,SILVER,Citation,21-1124.2(d2),Transportation Article,No,HISPANIC,M,ARLINGTON,VA,VA,A - Marked Patrol,"(38,9
82730733333;-77,1007551666667)"
12/01/2014,12:52:00,NCP,"6th district, Gaithersburg / Montgomery Village",FAILURE STOP AND YIELD AT THRU HWY,CHRISTOPHER
AVE/MONTGOMERY VILLAGE AVE,39,1628883333333,-77,2290883333333,No,No,No,No,No,No,No,No,No,No,No,MD,02 - Automobile,2001,HO
NDA,ACCORD,SILVER,Citation,21-403(b),Transportation Article,No,BLACK,F,UPPER MARLBORO,MD,MD,A - Marked Patrol,"(39,16288
833333;-77,22908833333)"
[dave@localhost traffic]$
```

And there were other problems. Many rows were too sparse, they were missing values in lots of fields. Rather than trying to guess what data should have been present in these cases, the rows with missing fields were just dropped. Here is the [script](#) that was used to set up the database, clean the incoming lines from the csv file, and then insert the rows into the database. After running the script to clean up the csv file, all but 1818 rows (about .1%) were successfully imported into the database.

```
[dave@localhost tmp]$ python fix-file.py | grep ERROR | wc -l
1818
[dave@localhost tmp]$ wc -l traffic.csv
1083708 traffic.csv
[dave@localhost tmp]$
```

Working with the data—determining a date range

How recent was this data? Did it cover a significant period of time? Some of the values for the year field were bad, so I had to define a range of data that looked reasonable before I could address the underlying question of ticket quotas. Since the file was last updated in

2015, 2015 was a clear upper bound. Any dates that we more recent had to be errors. At the lower end of the range, the number of rows per year dropped off sharply before 1994, but there were still significant numbers going back to the early 1990's. The period from 1990 to 2015 was chosen as most likely to hold accurate values.

```
sqlite> SELECT year, count(year) FROM alldata GROUP BY year;
0      809
4       3
5       1
6       3
--- SNIP ---
1986    778
1987   1119
1988   1509
1989   1689
1990   3005
1991   3659
1992   5651
1993   6881
1994  12017
1995  16694
1996  18729
1997  25766
1998  30632
1999  39528
2000  51483
2001  51493
2002  59170
2003  64478
2004  66794
2005  67255
2006  68922
2007  68032
2008  59877
2009  44734
2010  53482
2011  54611
2012  57094
2013  51225
2014  39097
2015  29480
2016  14522
2017   2022
2018     4
2019    12
2020     4
--- SNIP ---
9382     1
9510     1
9563     1
9867     1
9999    29
```

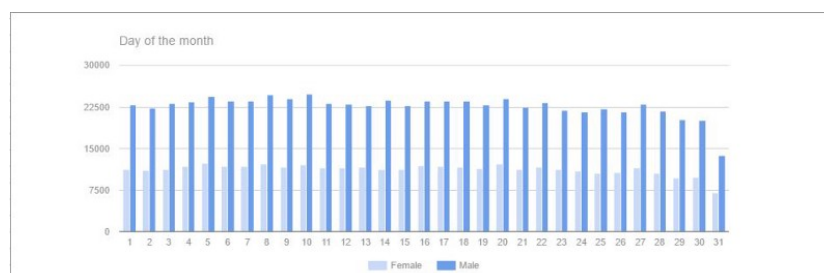

Parsing the date field into subfields

When the Maryland released the citation data, the date was reported as a single field using the format: 'mm/dd/yyyy'. That format was too coarse to answer my question. After adding additional columns for 'year', 'month', 'day', and 'day of the week' to the data tables, it was easy to get a more granular view of events by parsing the original date field during import to split it into component pieces.

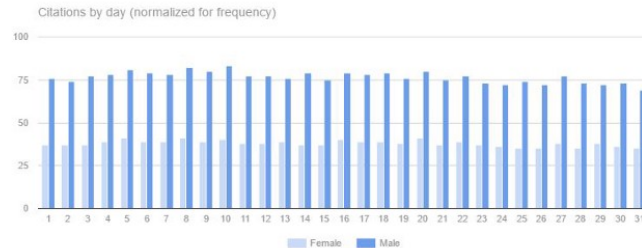
```
100 def parse_date(date_of_stop_field):
101     month, day, year = date_of_stop_field.split('/')
102     day_of_week = datetime.date(int(year), int(month), int(day)).weekday()
103
104     return "," + year + "," + month + "," + day + "," + str(day_of_week)
105
106
107 def parse_row(row):
108     embedded_comma = re.compile("(,)(\\w.*?)(,)(\\w.*)")
109     geo_coordinates = re.compile("(\\w)(\\w.*?)(,)(\\w.*)")
110     date_of_stop_field = re.compile("(\\d\\d)/(\\d\\d)/(\\d\\d\\d\\d)")
111
112     row = re.sub("\n", "", row) # Drop line break from row
113
114     with_commas = re.search(embedded_comma, row) # Find fields with internal commas
115     while(with_commas):
116         if (with_commas):
117             without_commas = with_commas.group(0).replace(",", "") # Remove internal commas
118             without_commas = "," + without_commas.replace("\n", "") + "," # Drop the quotes, return field separator
119             row = row.replace(with_commas.group(0), without_commas)
120             with_commas = re.search(embedded_comma, row) # Check for more quoted fields with commas
121
122     replacement = re.search(geo_coordinates, row) # Fix geo coordinate pairs
123     while (replacement):
124         row = re.sub(geo_coordinates, replacement.group(2) + "," +
125             + replacement.group(3), row, count=1)
126         replacement = ''
127
128     date_of_stop = re.search(date_of_stop_field, row)
129     if date_of_stop:
130         row = row + parse_date(date_of_stop.group(0)) # Give rows more date precision
131     else:
132         row = row + ",,,," # Keep number of fields consistent
133
134     return row
135
136 ...
```

Back to the question at hand ...

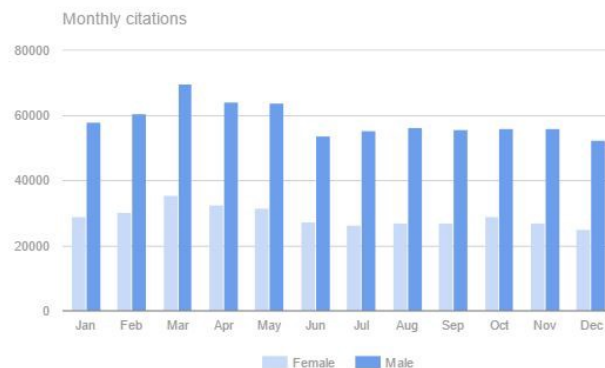
Now that it was possible to view the ticket data by day of the month, I could plot that to see if there were any obvious trends. If the police really do issue more citations at the end of the month to 'meet quota' then the the graph should show an uptick in reported citations as the month progress. However, a first glance shows just the opposite, a serious drop at the end of the month.



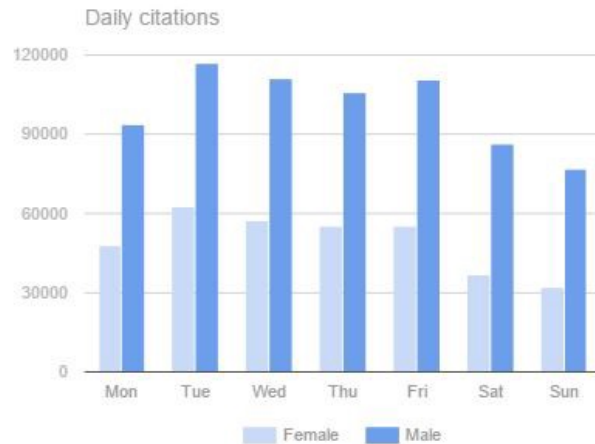
What's going on? Well one reason is that while all months have 28 days in them, only some months have 29 or more. Here is another view that has been adjusted to eliminate that artifact by showing the number of citations per month-day.



The second graph is more balanced in terms of tickets per day. Men hover around 76.5 citations per day over the 15 years from 1990 to 2015. Women average slightly more than half that figure, just 37.9 citations per day. Both averages are pretty consistent throughout the month, however, even with this frequency weighted view, the number of citations per day drops off towards the end of the month. That result is the opposite of popular opinion and supports the oft repeated statement from law enforcement that there are no quotas. “But”, asks the skeptic in the room, “perhaps there is a yearly quota?”



“Or a weekly one?”

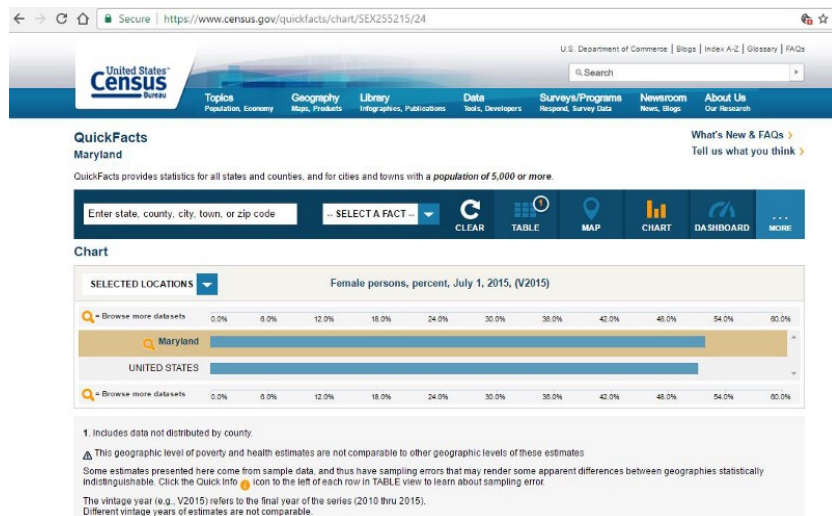


The data doesn't support either of those suggestions. In all cases: day of the month, day of the week, and month of the year, there are more citations given out toward the beginning of the period rather than towards the end. Given these figures it is hard to conclude that there is a last minute drive to meet a quota each month.

That settles the argument then. The data doesn't lie. Or does it? It turns out there are some difficulties with the data set.

Difficulties with the data

Even a cursory look at these numbers show men consistently getting twice as many citations as women over the entire 15 year period which is a surprising result. People may debate just how surprising it is that men get more tickets than women, but it is at least a little surprising that men would be cited twice as often as women. The [US Census website](#) reports that Maryland has a slightly large proportion of females (51.5%) than males (48.5%).



With roughly equal numbers of men and women, it seems reasonable to assume roughly equal numbers of male and female drivers. Perhaps there was an unreported selection process that skewed the data set. A few more queries and a closer look at the data indicate that there was, in fact, an additional selection criterion. All of the citations in the data set are related to traffic accidents.

```

traffic : sqlite3 - Konsole
File Edit View Bookmarks Settings Help
sqlite> SELECT count(Accident)
...> FROM alldata
...> WHERE (year > '1989') AND (year < '2001');
count(Accident)
-----
214045
sqlite> SELECT count(DateOfStop)
...> FROM alldata
...> WHERE (year > '1989') AND (year < '2001');
count(DateOfStop)
-----
214045
sqlite>

```

It is certainly interesting to see that men are ticketed in accidents twice as often as women, and that accident rates appear to be skewed towards the early parts of each week and month. Unfortunately since this data set only lists citations related to accidents and is silent on non-accident ticket rates, it means that this data set, as attractive as it may be, is insufficient to answer the original question. Additional data on non-accident related tickets is required to see if non-accident related citations are also issued at the same rate as accident related tickets. Sadly, we don't have that data.

Unanswered questions

So we don't have much insight into the original question, "are there ticket quotas". On the other hand we now have a few more topics for investigation. Do men really have twice as many accidents as women or do they just get ticketed twice as much for accidents they are involved in? Why are there spikes in the accident rate at the beginning of the week? Is there a significant number of accidents where tickets aren't issued? Is there a way to determine how many people and or vehicles were involved in a given accident? And perhaps the biggest question for this audience—where can researchers find the data needed to answer these questions?

Conclusion

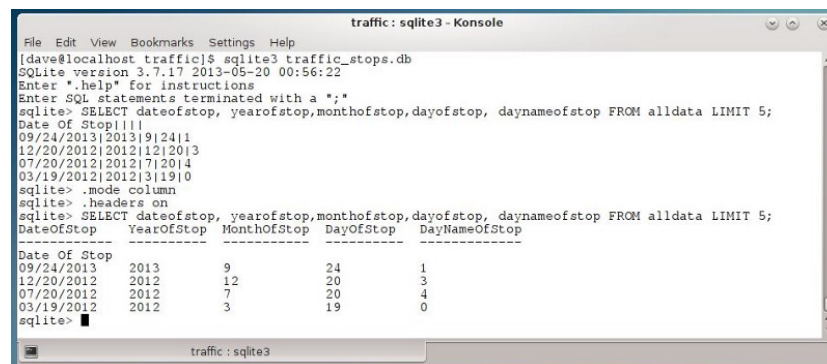
The Maryland data set is rich and can provide many interesting avenues of research. Unfortunately it only contains a subset of all citations issued and so it is insufficient to answer the question about quotas. However, working with this dataset has been a useful exercise, and it was especially valuable as an illustration of why it is critical to check data integrity, to clean incoming data, and to validate assumptions about the dataset before drawing conclusions from it.

Additional notes and details

For those curious about some of the more mechanical details of this investigation, the next sections discuss one or two points that didn't really fit in the story but that may be of some interest.

How to configure sqlite3 output

The default output style in sqlite3 works for some parsing tasks, but it is not ideal for all purposes. When exploring data it may be helpful to change the output to columnar form and to add a header row.



```
traffic: sqlite3 - Konsole
File Edit View Bookmarks Settings Help
[dave@localhost traffic]$ sqlite3 traffic_stops.db
SQLite version 3.7.17 2013-05-20 00:56:22
Enter ".help" for instructions
Enter SQL statements terminated with a ";"
sqlite> SELECT dateofstop, yearofstop, monthofstop, dayofstop, daynameofstop FROM alldata LIMIT 5;
Date Of Stop|||
09/24/2013|2013|9|24|1
12/20/2012|2012|12|20|3
07/20/2012|2012|7|20|4
03/19/2012|2012|3|19|0
sqlite> .mode column
sqlite> .headers on
sqlite> SELECT dateofstop, yearofstop, monthofstop, dayofstop, daynameofstop FROM alldata LIMIT 5;
DateOfStop      YearOfStop      MonthOfStop      DayOfStop      DayNameOfStop
-----
Date Of Stop
09/24/2013      2013           9           24           1
12/20/2012      2012          12           20           3
07/20/2012      2012           7           20           4
03/19/2012      2012           3           19           0
sqlite>
```

Use of SUM or COUNT in data assessment

COUNT works well when field values are easily filtered. On the other hand, if the values in the dataset are encoded as 1s and 0s then using SUM can be better quick way to get an aggregate count over a field. There are two benefits to using SUM like this. The SQL is more compact with SUM (there's no where clause) and rows are returned when the sum equals zero (COUNT doesn't return the output row in sqlite).

Here's an example counting alcohol related citations issued to Male drivers.

```
sqlite> SELECT year, COUNT(Alcohol)
...> FROM alldata
...> WHERE (year > '1989') AND (year < '2001')
AND gender = 'M' AND alcohol = 1
...> GROUP BY year;
Year      COUNT(Alcohol)
-----
1992      31
1993      42
1994      44
1996      44
1997      48
1998      35
1999      43
2000      103
sqlite> SELECT year, SUM(Alcohol)
...> FROM alldata
...> WHERE (year > '1989') AND (year < '2001')
AND gender = 'M'
...> GROUP BY year;
Year      SUM(Alcohol)
-----
1990      0
1991      0
1992      31
1993      42
1994      44
1995      0
1996      44
1997      48
1998      35
1999      43
2000      103
sqlite>
```

Thanks for reading this far, I hope these notes prove useful!

